

Архитектурный проект

KERNEL_PANIC

Data-Oriented Design

Все игровые сущности представлены одной POD-структурой EntityData. Никакой иерархии классов, виртуальных методов. Сущности хранятся в плоском массиве (`std::vector<EntityData>`), ссылки – по индексам. Обеспечивает локальность данных и предсказуемую производительность.

Реляционная модель через CSV

Все игровые параметры (локации, объекты карты, NPC, диалоги, предметы, скрипты, враги, группы врагов, фрагменты памяти, головоломки) вынесены во внешние CSV-файлы (папка `assets/`). Разделитель – точка с запятой `;`, текстовые поля в кавычках. Данные редактируются в Excel, игра перечитывает их при старте без перекомпиляции.

Разделение данных и поведения

- DataStore – контейнер всех загруженных данных и текущего состояния (сущности, игрок, инвентарь).
- Системы (ExplorationSystem, CombatSystem, DialogSystem, PuzzleSystem, RenderSystem, InputSystem) – пространства имён со статическими функциями. Они читают и модифицируют DataStore, но не хранят состояние.
- Состояния (ExplorationState, CombatState, DialogState, PuzzleState, GameOverState, FinalState) – управляют потоком игры, вызывают системы, переключаются по событиям. Состояния хранят только временный контекст (индексы врагов, строки диалога).

Инвентарь

Хранится в PlayerData как список `int` – идентификаторов скриптов из `scripts.csv`. Отображается на экране во всех состояниях, кроме боя (в ExplorationState, DialogState, PuzzleState, FinalState — виден; в CombatState скрыт). Игрок видит свои скрипты и может вспомнить названия. Проверка наличия скрипта перед применением в бою остаётся.

Боевая система

Пошаговая, работает через CombatSystem. Игрок вводит название скрипта, система ищет его в `scripts.csv`, проверяет инвентарь, вычисляет урон/эффекты (защита, оглушение, самоповреждение, блокировка скриптов). Враги атакуют последовательно. После боя – восстановление HP, добавление наград (скрипты, память). Интерфейс боя не показывает инвентарь, только поле ввода.

Память и нарратив

Процент памяти растёт после боссов и разговора с `healer.dll`. Влияет на выбор диалогов (таблица `dialogues.csv` с условиями по памяти и количеству фрагментов).

Игрок видит искажение логов через разные версии диалогов, ГГ никогда не осознаёт правду.

Линейный прогресс

Последовательность локаций фиксирована (`locations.next_location_id`). В локациях могут быть обязательные бои (`forced_combat_on_enter`), головоломки, боссы. Выход появляется после выполнения условий (собраны фрагменты, введён пароль, нажата последовательность).

Перезапуск (цикл)

В финале игрок выбирает ДА → сброс `PlayerData` (память 0%, инвентарь пуст, HP восстановлен) и переустановка `ExplorationState` с локацией `/ash` . Бесконечные циклы без изменения кода.

Технические стандарты

- C++17, CMake 3.16
- `unique_ptr` для состояний, индексы для сущностей – никаких `shared_ptr` в инвентаре.

Расширяемость

- Новые враги, предметы, диалоги, головоломки – добавление строк в CSV без компиляции.
- Новая логика поведения (например, новый `special_ai`) требует доработки `CombatSystem` и перекомпиляции – осознанное ограничение.

Архитектура полностью соответствует требованиям: ресурсные данные во внешних ассетах, отсутствие иерархий для сущностей, реляционная модель, DOD-подход, корректное управление памятью, а также корректное отображение инвентаря вне боя.

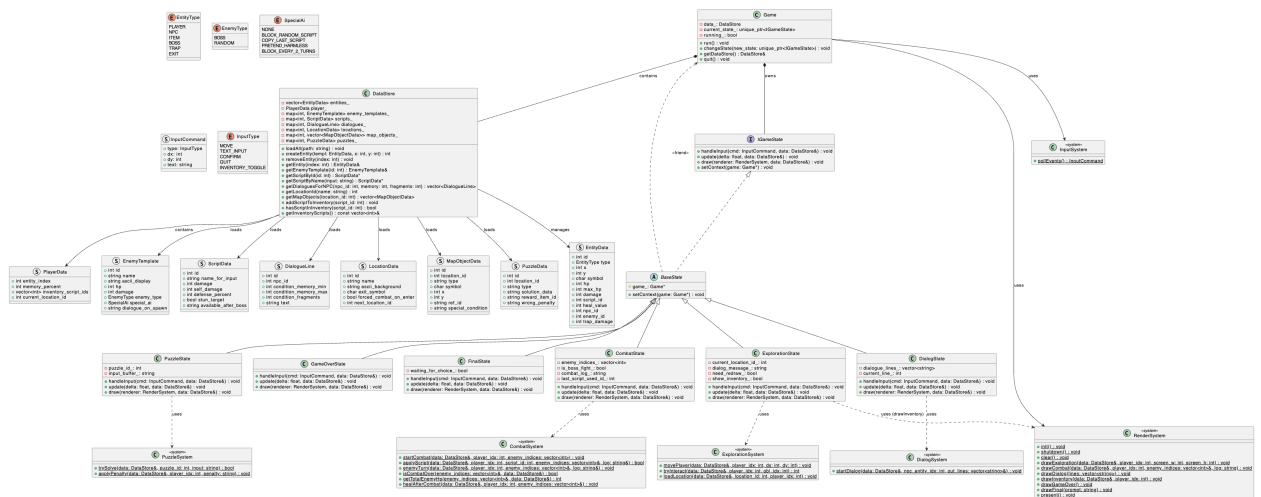


Рис. 1 Диаграмма классов

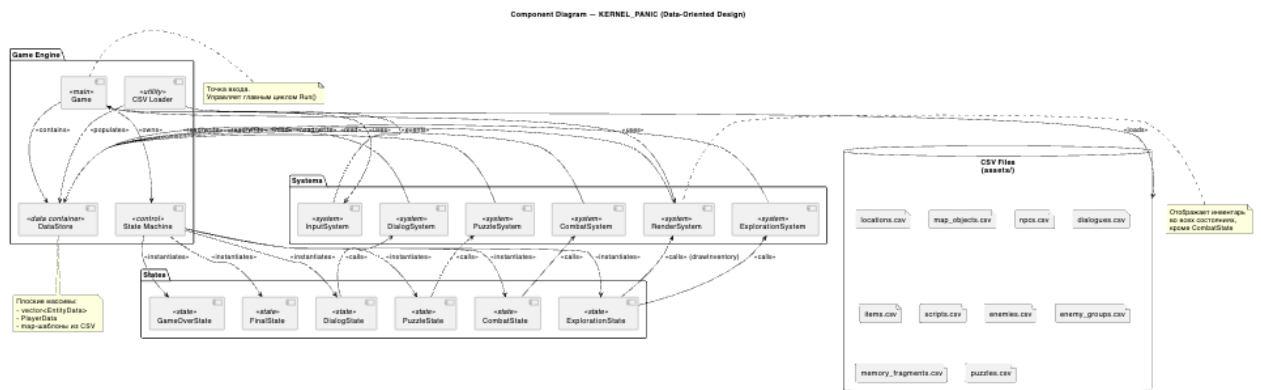


Рис. 2 Диаграмма компонентов

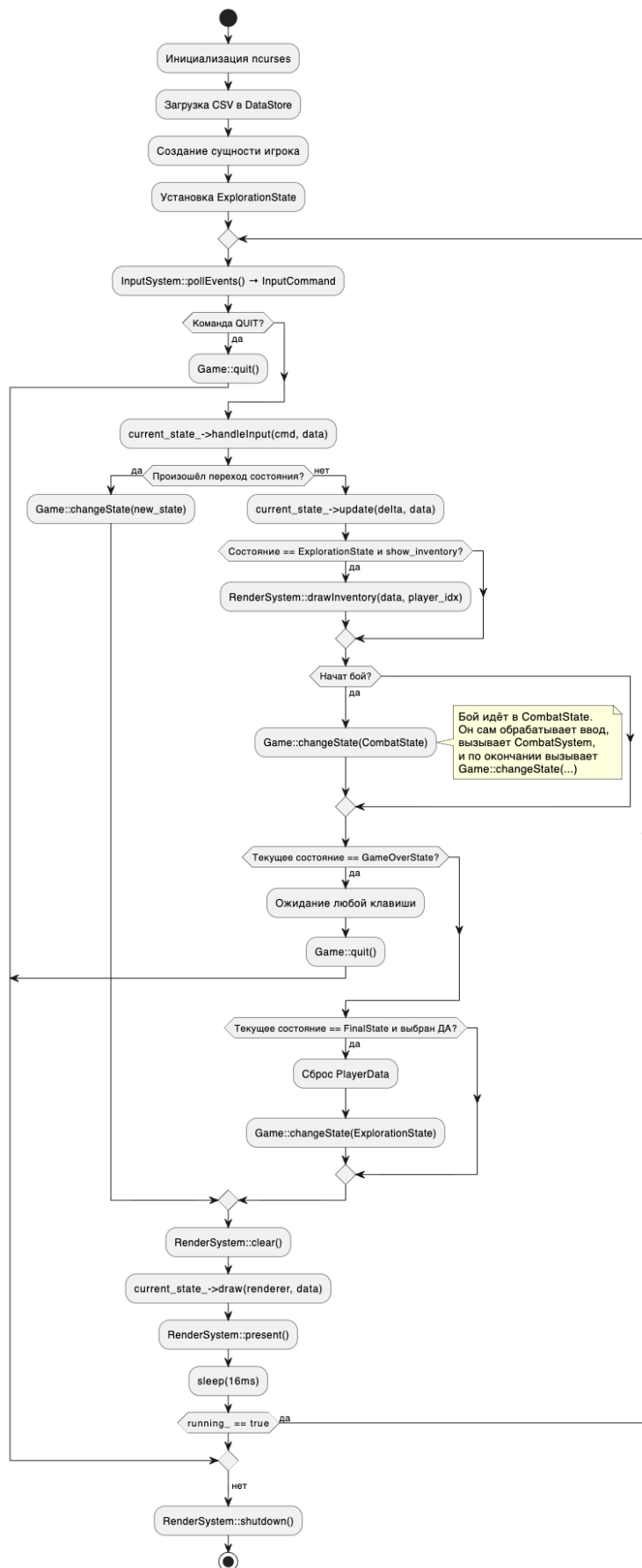


Рис. 3 Диаграмма активности

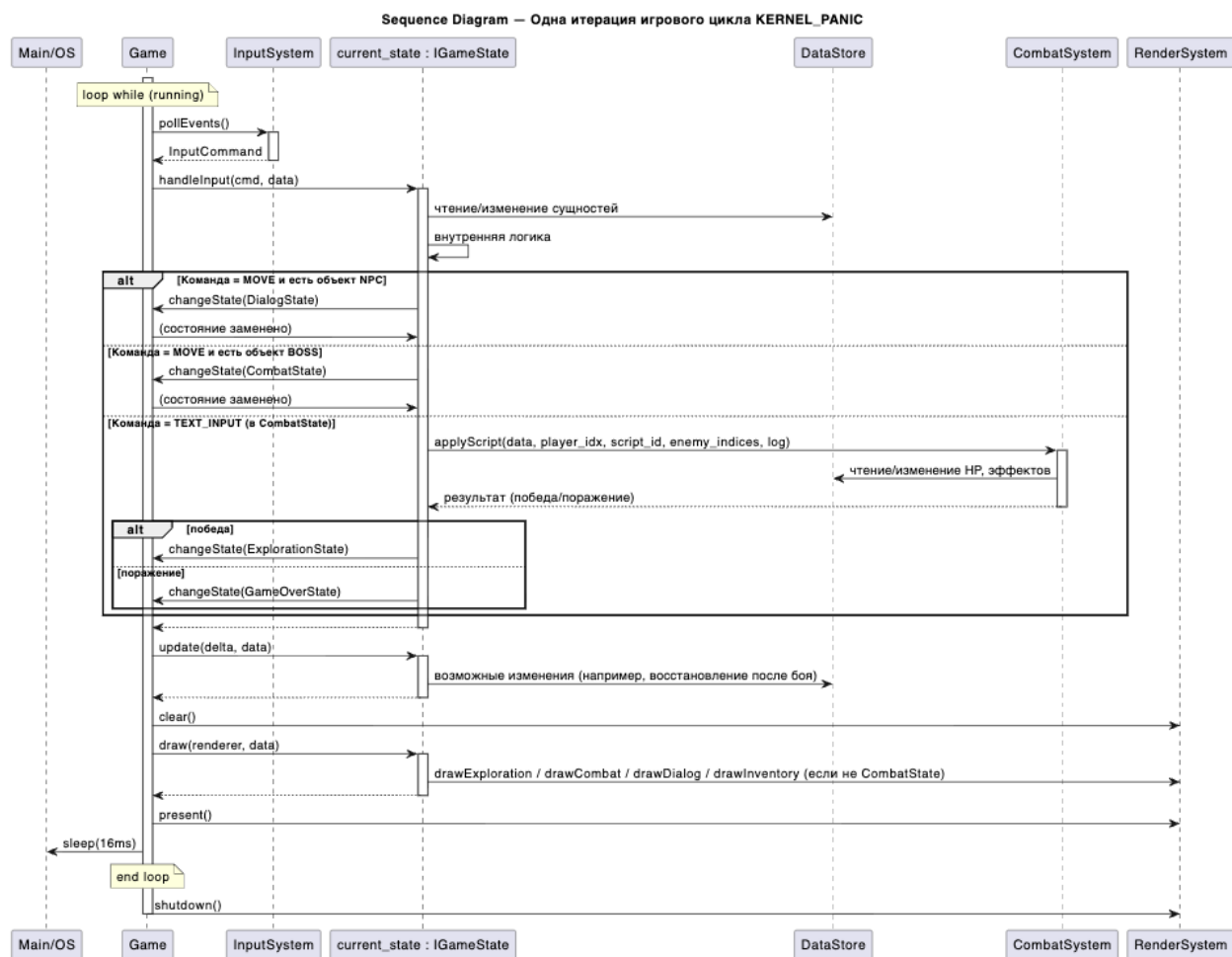


Рис. 4 Диаграмма последовательности

Управление потоком через состояния (State Machine)

Игра имеет шесть состояний: ExplorationState, CombatState, DialogState, PuzzleState, GameOverState, FinalState. Каждое состояние реализует интерфейс IGameState с методами HandleInput, Update, Draw. Состояния не хранят данные сущностей (всё в DataStore), только временный контекст: индексы врагов, строки диалога, ID головоломки. Переключение между состояниями происходит через Game::ChangeState().

ExplorationState – основной режим

- Отвечает за перемещение игрока по карте (ExplorationSystem::MovePlayer), взаимодействие с объектами (? , + , \$, боссы, выход).
- При входе в локацию загружает map_objects.csv для текущей location_id и создаёт сущности (DataStore::CreateEntity).
- Если у локации флаг forced_combat_on_enter = 1, сразу переключается в CombatState со случайной группой врагов из enemy_groups.csv.
- Отображает инвентарь (список скриптов) всегда, кроме боя – по нажатию специальной клавиши (например, I) или постоянно в боковой панели.

- При подборе предмета типа script добавляет script_id в PlayerData::inventory_script_ids ; при подборе heal мгновенно восстанавливает HP.
- После выполнения условий (собраны фрагменты логов, введён пароль, решена головоломка) активирует объект exit , переход к следующей локации по LocationData::next_location_id .

CombatState – пошаговый бой

- Создаётся с массивом индексов врагов (созданных через EnemyTemplate).
- Цикл боя: ожидание ввода строки → поиск скрипта в scripts.csv по name_for_input → проверка наличия в инвентаре (DataStore::HasScriptInInventory) → вызов CombatSystem::ApplyScript .
- После хода игрока – вызов CombatSystem::EnemyTurn , где враги атакуют по очереди, применяя свои SpecialAi (блокировка скриптов, притворство, копирование).
- При победе: CombatSystem::HealAfterCombat (фиксированное восстановление HP), добавление наград (скрипт, память через PlayerData::memory_percent), переход в ExplorationState .
- При поражении: переход в GameOverState .
- Интерфейс боя скрывает инвентарь, показывает поле ввода скрипта, суммарное HP врагов и ASCII-имя текущего врага.

DialogState и PuzzleState – временные режимы

- DialogState : загружает подходящие строки из dialogues.csv (фильтр по npc_id , memory_percent , количеству собранных фрагментов памяти). Показывает их поочерёдно; любое нажатие – возврат в предыдущее состояние.
- PuzzleState : для /tmp проверяет сбор трёх предметов fragment_log_ , затем активирует выход; для /xshaw проверяет ввод последовательности кнопок (решение из puzzles.csv). При ошибке в /xshaw – порождает временных врагов через переключение в CombatState . При успехе – даёт награду (reward_item_id) и открывает выход.

GameOverState и FinalState

- GameOverState : выводит синий экран с текстом "KERNEL_PANIC: фатальная ошибка. Причина: ты."; любая клавиша → завершение игры (Game::Quit).

- FinalState : активируется при входе в локацию / . Показывает зеркало и выбор [НЕТ] / [ДА] .

- НЕТ → выход из игры.

- ДА → сброс PlayerData : memory_percent = 0 , inventory_script_ids.clear() , HP восстанавливается до максимума, current_location_id = /ash , затем переход в ExplorationState . Игровой цикл повторяется бесконечно.

Системы (Systems) – статические функции без состояния

- ExplorationSystem : MovePlayer , TryInteract , LoadLocation .
- CombatSystem : ApplyScript , EnemyTurn , HealAfterCombat , GetTotalEnemyHp .
- DialogSystem : StartDialog .
- PuzzleSystem : TrySolve , ApplyPenalty .
- RenderSystem : Init , Shutdown , Clear , DrawExploration , DrawCombat , DrawDialog , DrawInventory , DrawGameOver , DrawFinal , Present .
- InputSystem : PollEvents .

Поток выполнения (главный цикл Game::Run)

- Инициализация ncurses (RenderSystem::Init), загрузка CSV (DataStore::LoadAll), создание игрока (DataStore::CreateEntity).

- Установка ExplorationState .

- Бесконечный цикл:

- Опрос ввода (InputSystem::PollEvents) → current_state_->HandleInput .

- При необходимости смена состояния через ChangeState .

- current_state_->Update (обычно пусто, кроме CombatState).

- RenderSystem::Clear , current_state_->Draw , RenderSystem::Present .

- Задержка 16 мс.

- При завершении – RenderSystem::Shutdown .

Инвентарь и память – кросс-состояний

- Инвентарь отображается во всех состояниях, кроме CombatState (по требованию GDD). В ExplorationState можно открыть окно инвентаря по клавише I, либо он всегда виден в боковой панели.

- Память (PlayerData::memory_percent) влияет на выбор диалогов и на финальный текст. После победы над боссом watcher (+30%), rack (+30%), liar (+25%) и после разговора с healer.dll (+10%) – итоговые 95% перед финалом.

- Фрагменты памяти (пять штук в /recover) собираются в инвентарь как обычные предметы типа memory_frag . Они не дают скриптов, но меняют диалог с healer.dll .

Взаимодействие с DataStore

- Все системы и состояния работают с DataStore через ссылку, передаваемую в методы.

- DataStore предоставляет методы для чтения/записи сущностей, инвентаря, памяти, а также для доступа к загруженным шаблонам (враги, скрипты, диалоги).

- Никакие глобальные переменные не используются.

Соответствие DOD и требованиям

- Состояния и системы разделены: состояния управляют потоком, системы – чистой логикой.

- Нет виртуальных методов в сущностях, нет иерархий.

- Переключение состояний происходит через std::unique_ptr, без утечек.

- На macOS используется ncurses, код систем не зависит от платформы.

- Все методы и функции названы в PascalCase, поля классов – snake_case_ с подчеркиванием в конце (Google C++ Style).

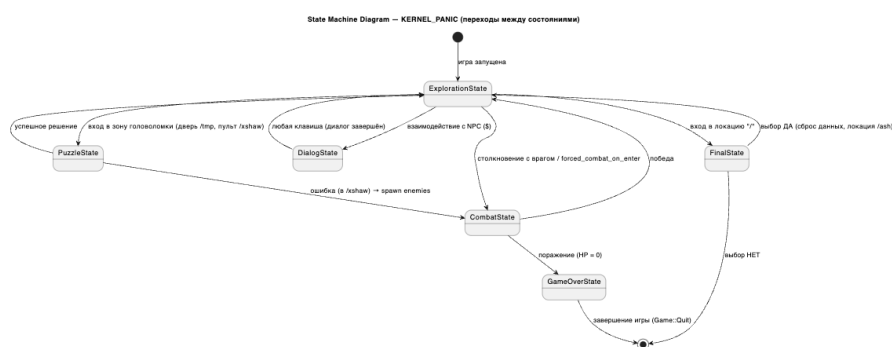


Рис. 5 Диаграмма переходов между состояниями

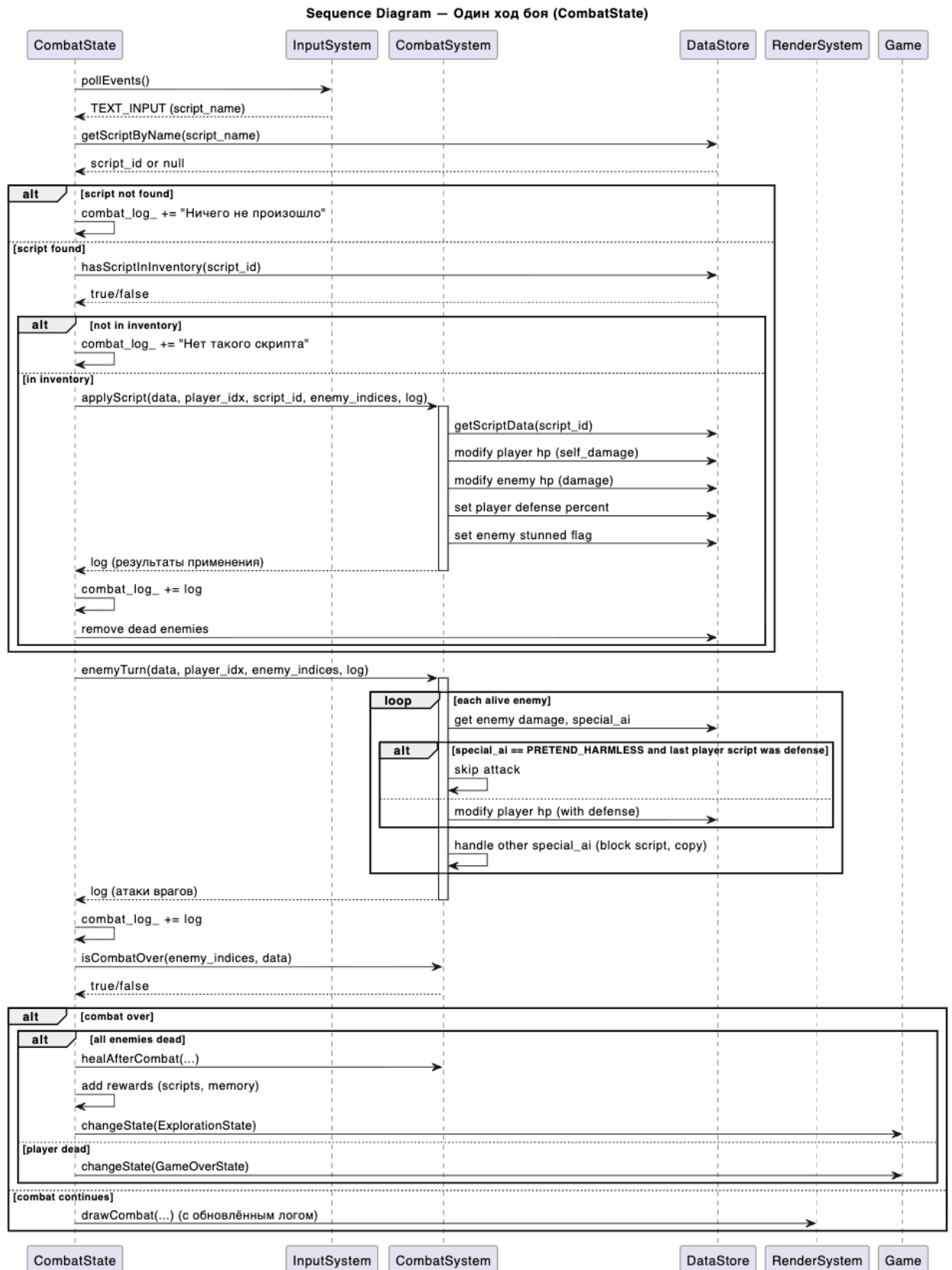


Рис. 6 Диаграмма последовательности боя

Рендеринг и ввод на macOS (ncurses)

Выбор библиотеки – ncurses

Для работы с консолью на macOS используется библиотека `ncurses`. Она предоставляет независимые от терминала функции: управление курсором, цветом (хотя у нас чёрно-белый интерфейс), чтение клавиш без ожидания Enter, очистка экрана, позиционирование. Альтернатива (ANSI-escape последовательности + `termios`) сложнее и менее надёжна.

Инициализация и завершение

- `initscr()` – инициализация `ncurses`.
- `cbreak()` – отключение буферизации ввода (клавиши передаются сразу).
- `noc_echo()` – не выводить нажатые клавиши на экран.
- `keypad(stdscr, TRUE)` – включение поддержки стрелок, функциональных клавиш.
- `nodelay(stdscr, FALSE)` – блокирующий ввод (ждём нажатия). Для таймингов можно сделать `nodelay(TRUE)` и `napms(16)` в цикле, но проще использовать блокирующий `getch()` с отдельным циклом обновления.
- `endwin()` – завершение работы `ncurses`.

RenderSystem – статические функции

Все функции отрисовки статические, принимают `DataStore&` и индексы/параметры.

- `init()` – вызывает `initscr()` и настройки.
- `shutdown()` – вызывает `endwin()`.
- `clear()` – `clear()` из `ncurses`.
- `drawExploration(data, player_idx, screen_w, screen_h)` – отрисовка карты:
 - Полосы HP/MEM в верхней строке (используя символы ■ и пробелы, проценты).
 - Центральная область: ограниченная область вокруг игрока (например, 21x21 клетка). Для каждой видимой клетки вызывается `mvaddch(y, x, symbol)`.
 - Окно диалога внизу: рамка из # через `border()` или ручное рисование, внутри текст `dialog_message_`.
 - Инвентарь: если `show_inventory == true` (в `ExplorationState`), отображается список скриптов в правой части экрана (или отдельным окном).
- `drawCombat(data, player_idx, enemy_indices, log)` – боевой экран:
- Верхняя строка: HP игрока (слева) и суммарное HP врагов (справа).

- Центр: фиксированный символ @ на фиксированной позиции (например, 40, 12).
- Ниже – крупное ASCII-имя текущего врага (первого живого) – выводится через `printw` или последовательно `mvdaddch`.
- Нижнее окно диалога – лог боя и приглашение к вводу (>).
- `drawDialog(lines)` – отображает диалоговое окно на весь экран или поверх карты (затемнение не требуется).
- `drawInventory(data, player_idx)` – рисует список скриптов из инвентаря (только в `ExplorationState`, `DialogState`, `FinalState`, но не в `CombatState`).
- `drawGameOver()` – синий экран с текстом и рамкой (используем `COLOR_BLUE` если поддерживается, или просто синий фон через `init_pair`).
- `drawFinal(prompt)` – зеркало (# рамка, пустота внутри), текст выбора.
- `present()` – `refresh()` для вывода на экран.

InputSystem – обработка клавиш

- `pollEvents()` – блокирующий вызов `getch()`. Возвращает `InputCommand`.
- Преобразование:
 - Стрелки / WASD → `InputType::MOVE` с `dx`, `dy`.
 - I или i → `InputType::INVENTORY_TOGGLE` (показ/скрытие инвентаря в `ExplorationState`).
 - Enter → переключение в режим ввода строки: `echo()` + `curs_set(1)` + `getstr()` + `noecho()` + `curs_set(0)`, возвращает `TEXT_INPUT` с текстом.
 - Q → `QUIT`.
 - Цифры / буквы в головоломках – обрабатываются как `TEXT_INPUT` или отдельные команды в `PuzzleState`.

Особенности macOS

- Библиотека `ncurses` обычно предустановлена, но в CMake нужно добавить `find_package(Curses REQUIRED)` и линковать `${CURSES_LIBRARIES}`.
- Для работы с мышью (не требуется) – игнорируем.
- Русские символы (в диалогах) поддерживаются, если файл в UTF-8 и терминал настроен соответствующим образом. ASCII-имена врагов (`WATCHER` , `RACK`) – латиница.

- Для синего экрана смерти можно использовать `init_pair(1, COLOR_WHITE, COLOR_BLUE)` и `attron(COLOR_PAIR(1))`.

Управление инвентарём (отображение)

- По нажатию `I` в `ExplorationState` переключается флаг `show_inventory_`.
- При `show_inventory_ == true`, `RenderSystem::drawExploration` дополнительно рисует окно инвентаря (например, справа или снизу).
- Инвентарь читается из `DataStore::getInventoryScripts()` и по `id` получает название скрипта из `scripts.csv`.
- В `CombatState` инвентарь не показывается, даже если флаг включён – это требование GDD.

Кросс-платформенность с Windows

Если потребуется портировать на Windows, можно использовать `pdccurses` (совместим с `ncurses` API) или написать отдельную реализацию через `windows.h`. В архитектуре системы рендеринга и ввода изолированы, поэтому замена библиотеки не затронет остальной код.

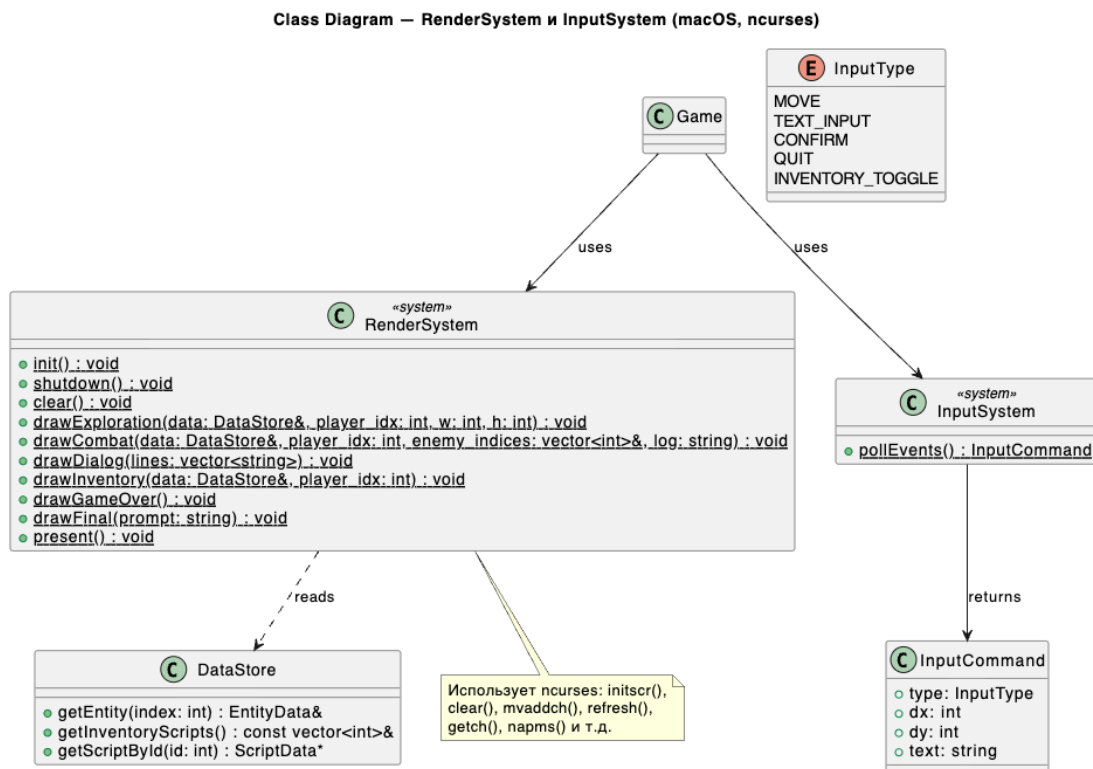


Рис. 7 Диаграмма классов рендеринга

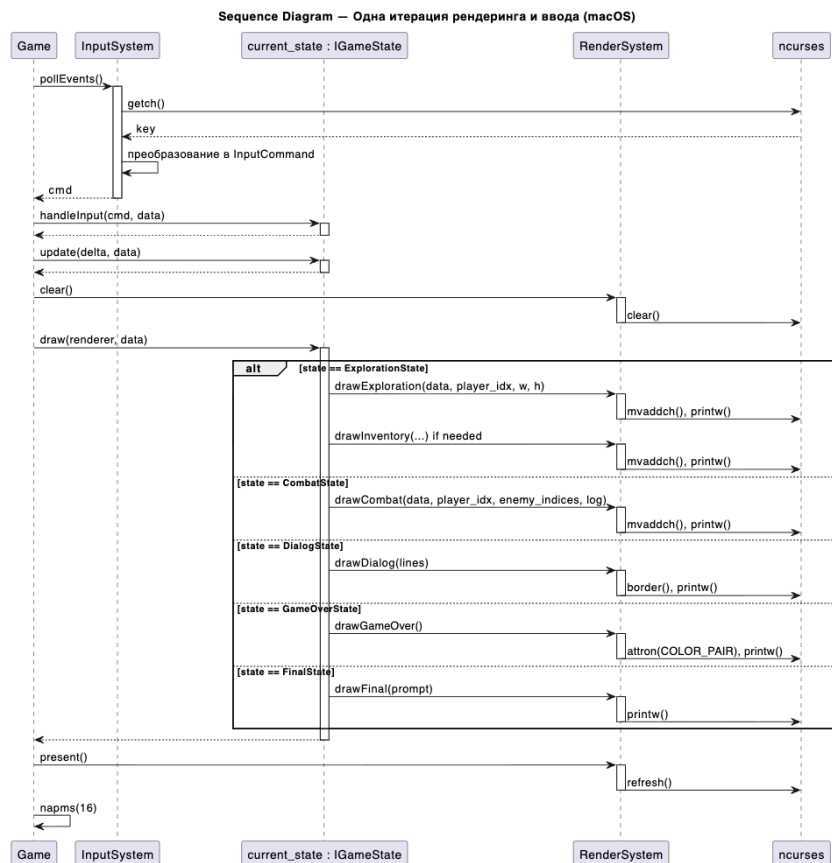


Рис. 8 Диаграмма итерации рендеринга

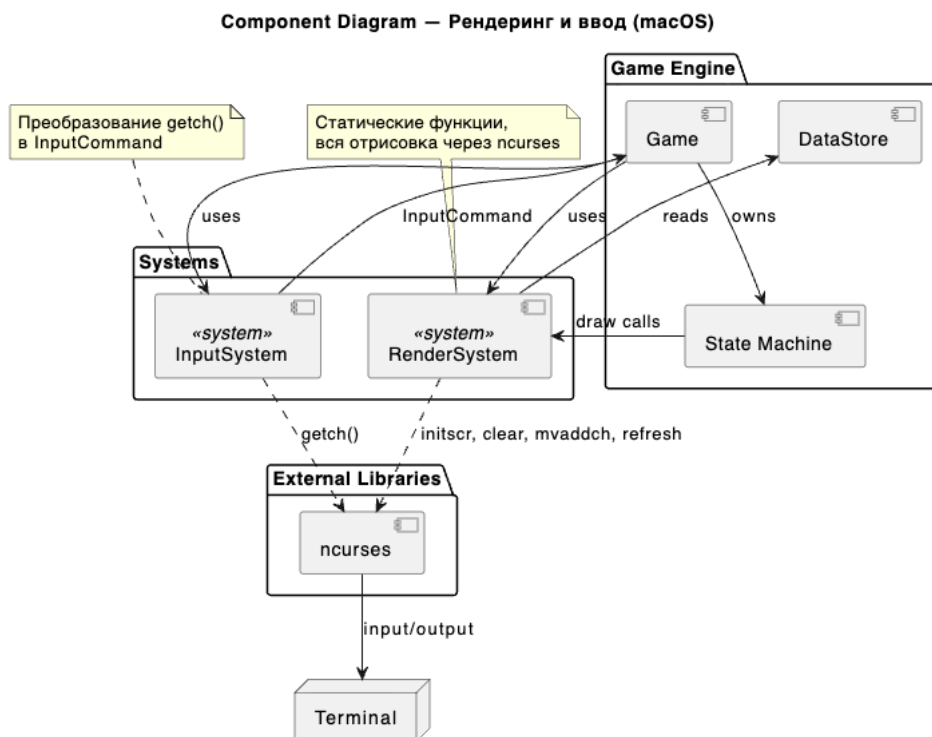


Рис. 9 Диаграмма компонентов подсистемы рендеринга

Диалоги, головоломки, память, финал и управление данными (CSV)

Диалоговая система (DialogSystem)

- Статическая функция `startDialog(data, npc_entity_idx, out_lines)` загружает подходящие реплики из `dialogues.csv`.
- Фильтрация по: `npc_id`, `condition_memory_min` $\leq$ `player.memory_percent` $\leq$ `condition_memory_max`, `condition_fragments` $\leq$ `collected_fragments` (количество фрагментов памяти, собранных игроком в `/recover`).
- Если подходит несколько диалогов, берётся тот, у которого наиболее строгие условия (максимальный `condition_memory_min` или `condition_fragments`).
- `DialogState` отображает строки по очереди, после прочтения возвращает управление в предыдущее состояние.
- Реплики могут содержать намёки на цикличность (например, «ты уже видел это»). Текст диалога может меняться в зависимости от памяти, создавая искажение логов.

Головоломки (PuzzleSystem)

- Поддерживаемые типы головоломок (из `puzzles.csv`):
 - `collect_items` – для `/tmp`: требуется собрать определённые предметы (указаны в `solution_data` через запятую). После сбора активируется выход.
 - `button_order` – для `/xshaw`: игрок вводит последовательность цифр (решение в `solution_data`). При совпадении – выход и награда.
 - `PuzzleSystem::trySolve(data, puzzle_id, input)` проверяет решение. При успехе: удаляет ненужные объекты, активирует `exit` в текущей локации, выдаёт награду (`reward_item_id`). При ошибке: вызывает `applyPenalty` (например, спавнит временных врагов через `CombatSystem`).
- Головоломка `/recover` – сбор пяти фрагментов памяти (`memory_fragments.csv`). Они не влияют на прохождение, но открывают особый диалог у `healer.dll` (если собраны все 5). Фрагменты хранятся в `PlayerData` как счётчик `collected_fragments`.

Система памяти (memory mechanic)

- `PlayerData.memory_percent` изменяется:
 - +30% после `watcher.eze`
 - +30% после `rack.eze`
 - +10% после разговора с `healer.dll`

+25% после liar.eze

Итог 95% перед финалом.

- Память влияет на выбор диалогов (искажение логов) и может использоваться для изменения поведения (например, в будущем можно добавить разблокировку скриптов).

- В финале при 95% игроку предлагается выбор. Если игрок выбирает ДА, вызывается `PlayerData::resetForNewCycle()` : обнуление памяти, очистка инвентаря, восстановление HP, сброс текущей локации на /ash . Game переустанавливает `ExplorationState` . Это создаёт бесконечный цикл без изменения кода.

Финальное состояние (FinalState)

- При входе в локацию / (нет врагов, нет выхода) активируется `FinalState` .

- Отображается зеркало (рамка из # с пустотой внутри) и текст:

я почти все вспомнил. 95% памяти. осталось завершить миссию.

ты убил всех. ты забыл все. ты свободен.

хочешь вспомнить? [НЕТ] [ДА]

- Ввод НЕТ → `Game::quit()` . Ввод ДА → сброс и перезапуск (см. выше).

Управление данными (DataStore и CsvLoader)

- `DataStore` – центральное хранилище:

- `vector<EntityData> entities_` – все сущности (игрок, NPC, предметы, враги, боссы, ловушки).

- `PlayerData player_` – данные игрока (индекс сущности, память, инвентарь, текущая локация).

- Словари-шаблоны: `enemy_templates_` , `scripts_` , `dialogues_` , `locations_` , `map_objects_` (по `location_id`), `puzzles_` .

- `CsvLoader` – загрузка всех CSV из `assets/` с разделителем ; , поддержка кавычек, преобразование строк в числа/enum. При старте вызывает `DataStore::loadAll()` .

- Валидация внешних ключей: проверка, что `ref_id` в `map_objects` существует в соответствующей таблице. При ошибке – вывод сообщения и аварийное завершение (или игнорирование строки).

- Индексация: после загрузки для быстрого доступа строятся unordered_map (например, map<string, int> location_name_to_id , map<string, int> script_name_to_id).

Пример загрузки локации в ExplorationState

- При смене локации через loadLocation получаем location_id .
- Очищаем текущую карту (удаляем все сущности, кроме игрока).
- Читаем map_objects[location_id] и для каждой записи вызываем DataStore::createEntity с параметрами из CSV (тип, символ, позиция, ref_id для подстановки шаблона).
- Если forced_combat_on_enter == 1, немедленно переключаемся в CombatState со случайной группой из enemy_groups для этой локации.

Интеграция с боевой системой и инвентарём

- При победе над боссом CombatState вызывает DataStore::addScriptToInventory(reward_script_id) и увеличивает player.memory_percent .
- При подборе предмета типа script в ExplorationState – аналогично добавляется скрипт.
- Предметы лечения (+) мгновенно восстанавливают HP и удаляются.
- Фрагменты памяти (memory_frag) увеличивают счётчик collected_fragments в PlayerData , но не дают скриптов.

Цикличность и перезапуск

- После выбора ДА в финале сбрасываются только PlayerData , но не шаблонные данные (CSV). Это позволяет проходить игру заново с нулевой памятью и пустым инвентарём.
- Все NPC и боссы восстанавливаются, так как их состояние не хранится (кроме удалённых временных сущностей).
- Игрок видит те же диалоги и события, но может заметить повторения (например, фраза healer.dll «ты уже видел это»). Это является единственным прямым намёком на цикл.

Расширяемость через CSV

- Без перекомпиляции:
- Добавление новых диалогов (условия по памяти и фрагментам).
- Добавление новых головоломок существующих типов (новые `collect_items` или `button_order`).
- Добавление новых фрагментов памяти (меняется только текст).
- Изменение последовательности локаций (`next_location_id`), обязательных боёв.
- С перекомпиляцией (редко):
- Новый тип головоломки (требует реализации в `PuzzleSystem`).
- Новое условие для диалога (например, `has_item`).
- Изменение логики сброса/перезапуска.

Резюме по блоку 5

- Диалоги, головоломки и память полностью управляются данными (CSV) и не требуют изменения кода для контентных правок.
- `DataStore` с `CsvLoader` обеспечивают централизованную загрузку и валидацию, а также индексацию для быстрого доступа.
- Финал и перезапуск реализованы через сброс `PlayerData` и смену состояния – чистый цикл без накопления ошибок.
- Архитектура сохраняет DOD-подход: данные отделены от поведения, системы работают с плоскими массивами.

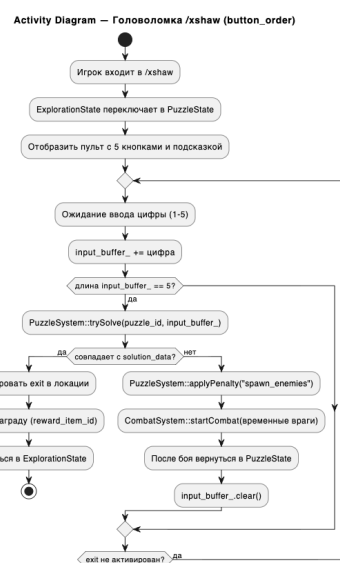


Рис. 10 Диаграмма активности для головоломки

State Machine — Память и финал

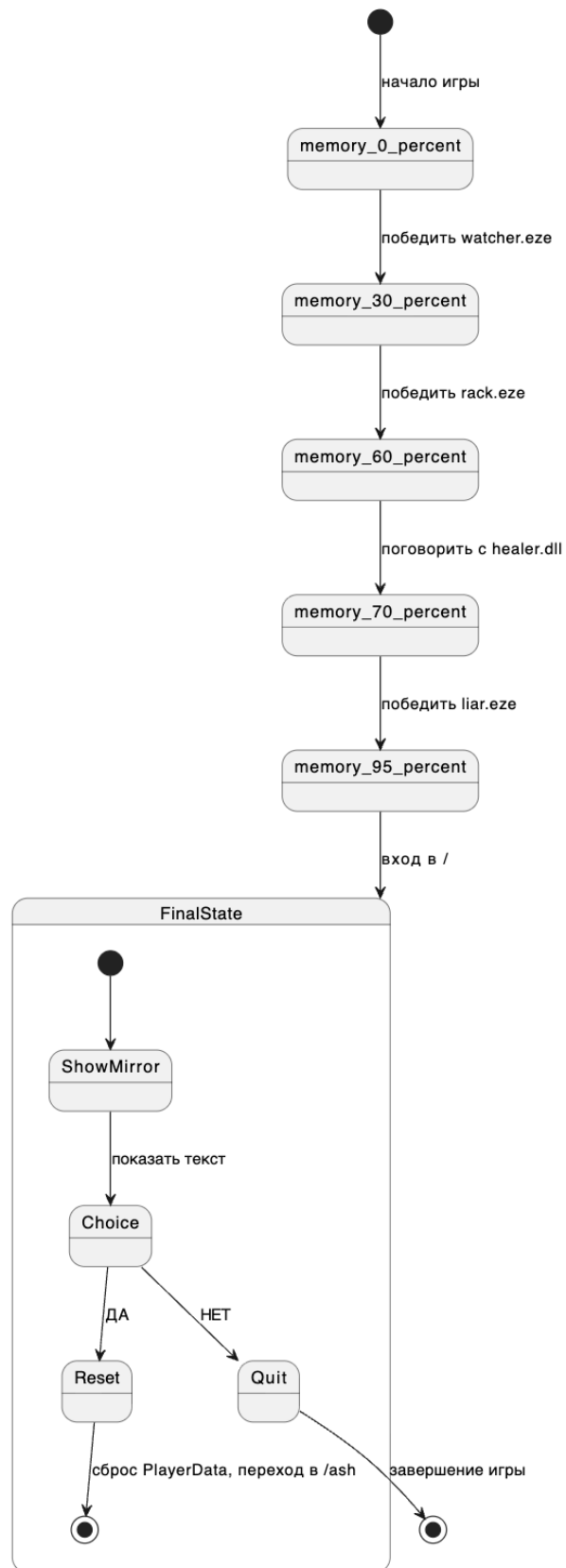


Рис. 11 Диаграмма изменения уровня памяти

Class Diagram — DataStore и CsvLoader

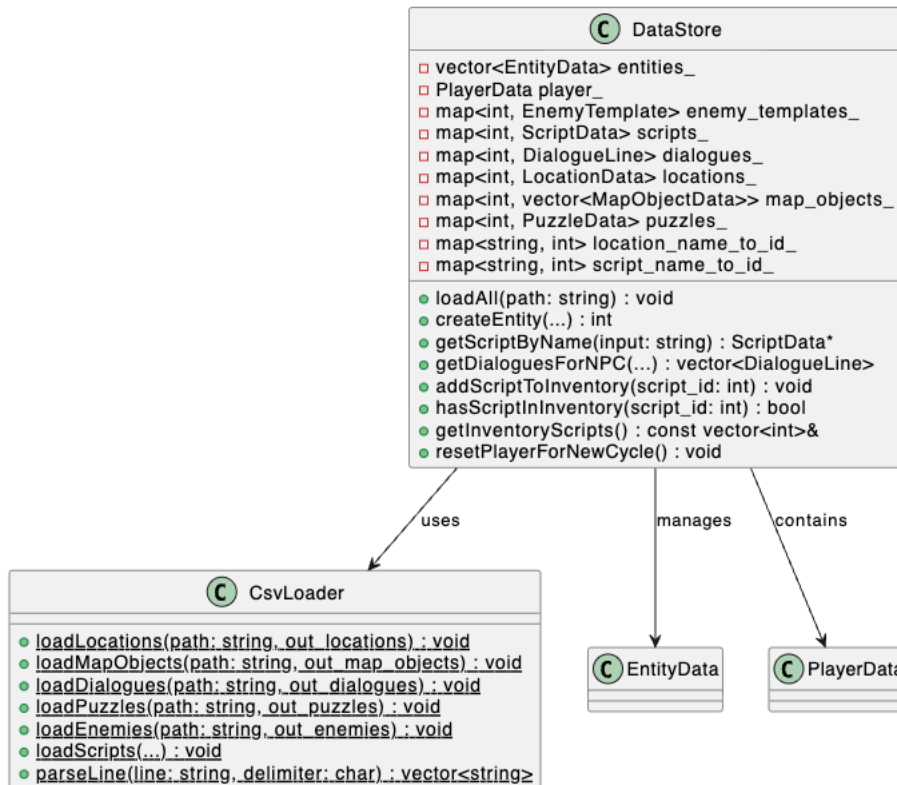


Рис. 12 Диаграмма классов работы с данными

ER Diagram — Связи CSV таблиц

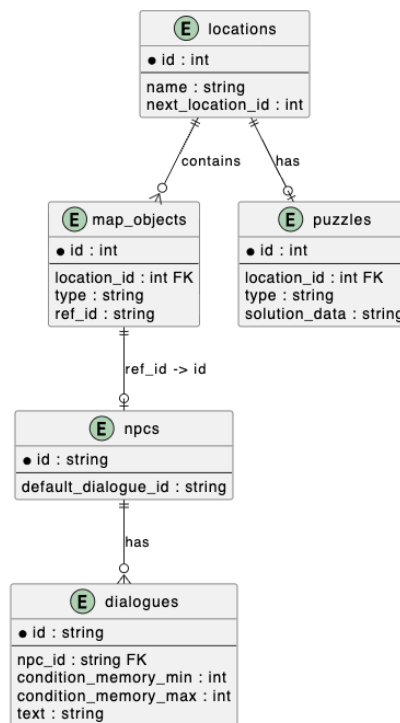


Рис. 13 Диаграмма связей между таблицами